

Actividad de Laboratorio: Arquitectura Multi-Nivel (N-Tier) y Patrón Lógico de Software (MVC) en .NET

1. El Tránsito hacia los Sistemas Distribuidos y Multi-Capa

Limitación del Monolito Local:

- Sincronización: los datos no pueden ser compartidos entre múltiples usuarios simultáneamente sin generar conflictos de concurrencia
- Escalabilidad: Para aumentar la capacidad, se debe actualizar el hardware, lo cual tiene un límite físico y económico.
- Disponibilidad: Si la máquina falla, todo el sistema queda operativo.
- Mantenimiento: Actualizar cualquier componente requiere detener todo el sistema.
- Colaboración: Múltiples desarrolladores no pueden trabajar simultáneamente en diferentes partes sin conflictos

Distinción Crítica(Layers vs Tiers)

- Capas Lógicas(Layers): Son la organización conceptual del código dentro de una aplicación. Representan la separación de responsabilidades a nivel de diseño y desarrollo. Estas capas pueden residir en un mismo servidor físico.
- Niveles Físicos (Tiers): Son la distribución física del software en diferentes máquinas o servidores. Cada tier ocupa un hardware separado y se comunican a través de la red. Representan el despliegue real de la aplicación.

Responsabilidades en la Arquitectura de 3 Niveles

Nivel 1: Capa de Presentación

- Misión exclusiva: Gestionar la interacción con el usuario, mostrar la interfaz y capturar las entradas del usuario.
- Tecnología común: HTML, CSS, JavaScript, frameworks frontend (React, Angular, Vue), ASP.NET Razor Pages.

Nivel 2: Capa de Aplicación o Negocio (Application Tier)

- Misión exclusiva: Contener la lógica de negocio, procesar las reglas de la aplicación, validar datos y coordinar las operaciones.
- Tecnología común: C#, Java, Python, Node.js, servidores de aplicaciones (IIS, Apache Tomcat, etc.).

Nivel 3: Capa de Datos (Data Tier)

- Misión exclusiva: Almacenar, recuperar y gestionar los datos de manera persistente.
- Tecnología común: SQL Server, MySQL, PostgreSQL, Oracle, bases de datos NoSQL (MongoDB, Redis).

Seguridad Perimetral:

Es un error crítico exponer el puerto de una base de datos a Internet porque:

- La base de datos es el activo más valioso del sistema (contiene toda la información sensible).
- Los sistemas de gestión de bases de datos no están diseñados para resistir ataques directos desde Internet.
- Aumenta dramáticamente la superficie de ataque.
- Las bases de datos suelen tener credenciales más débiles o por defecto.

Buena práctica recomendada: Colocar la base de datos en una red privada (VPC o red interna), accesible solo desde el servidor de aplicaciones mediante un firewall, utilizando redes privadas virtuales (VPN) o conexiones cifradas, y nunca exponiendo los puertos de la base de datos directamente a Internet.

2. Desacoplamiento Lógico con el Patrón MVC

La Crisis de código espagueti

Mezclar sentencias SQL, lógica matemática y etiquetas visuales en un mismo archivo genera:

- Difícil mantenimiento: Cambiar un aspecto puede romper otros sin relación aparente
- Pruebas unitarias imposibles: No se puede aislar la lógica de negocio para probarla
- Baja reusabilidad: El código no puede ser aprovechado en otros contextos
- Alta complejidad cognitiva: Un solo archivo requiere entender múltiples dominios
- Errores difíciles de rastrear: Problemas de presentación, lógica o datos se mezclan
- Colaboración limitada: Varios desarrolladores no pueden trabajar en el mismo archivo

Separación de Preocupaciones (SoC):

Modelo:

- Qué representa: Los datos y la lógica de negocio de la aplicación. Encapsula el estado y las reglas del dominio.
- Por qué no debe conocer cómo se muestran los datos: Para mantener la independencia y reusabilidad. El modelo puede ser utilizado por diferentes

vistas (web, móvil, API) sin modificaciones. Al ignorar la presentación, el modelo se enfoca únicamente en la lógica de negocio y persistencia.

Vista:

- Por qué se define como entidad pasiva e inteligente:

Pasiva: No contiene lógica de negocio, solo recibe datos y los presenta

Inteligente: Sabe cómo mostrar los datos de manera óptima, puede tener lógica de presentación (formateo, condicionales de visualización) sin involucrar lógica de negocio

- Código estrictamente prohibido: No debe contener lógica de negocio, acceso a bases de datos, operaciones de cálculo complejas, ni manipulación directa del modelo

Controlador:

- Rol como intermediario táctico y director de orquesta:

Recibe las peticiones del usuario

Coordina la interacción entre el Modelo y la Vista

Orquesta el flujo de la aplicación (qué modelo usar, qué vista mostrar)

No contiene lógica de negocio pesada, solo actúa como "pegamento" entre componentes

Gestiona el enrutamiento y la validación básica de entrada.

Métricas de Ingeniería de Software:

Alta Cohesión: Cada componente MVC tiene una responsabilidad única y bien definida:

Modelo: Datos y lógica de negocio

Vista: Presentación

Controlador: Orquestación

Esto permite que cada componente sea más robusto y fácil de mantener.

Bajo Acoplamiento: Los componentes se comunican a través de interfaces bien definidas:

La Vista no conoce el Modelo directamente

El Modelo no sabe nada de la Vista

El Controlador actúa como intermediario

Esto permite modificar un componente sin afectar a los demás, facilitando pruebas, mantenimiento y evolución del sistema.

2: Modelado del Ciclo de Vida y Enrutamiento Semántico

1. Mapeo Analítico de URLs

URL Entrante del Cliente	Clase Controladora Buscada por el Framework	Método (Acción) Ejecutado	Parámetro id Inyectado
https://ingenieria.usac.edu.gt/ControlAcademico/Login	ControlAcademico Controller	Login	(ninguno - sin id)
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10	AsignacionController	Detalle	10
https://ingenieria.usac.edu.gt/Home	HomeController	Index	(ninguno - id opcional no enviado)

2. Diagramación del Flujo Interactivo

Viaje completo de una petición HTTP:

Paso 1 - Navegador del Usuario:

El usuario hace clic en un botón o enlace en su navegador. El navegador construye una petición HTTP (GET, POST, etc.) con la URL correspondiente y la envía al servidor web.

Paso 2 - Servidor Web y Middleware:

La petición llega al servidor (IIS/Kestrel). El middleware de ASP.NET Core procesa la petición a través de la pipeline de middleware (autenticación, autorización, logging, etc.). El motor de enrutamiento analiza la URL y determina qué Controlador y Acción deben manejar la petición.

Paso 3 - Controlador (Director de Orquesta):

El framework instancia el Controlador correspondiente y ejecuta el método de Acción especificado. El Controlador:

- Recibe los parámetros de la URL
- Valida la entrada (validación básica)
- Solicita datos al Modelo (si es necesario)
- No ejecuta lógica de negocio compleja

Paso 4 - Modelo (Lógica de Negocio):

Si la acción requiere datos, el Controlador interactúa con el Modelo:

- El Modelo obtiene/actualiza los datos desde la fuente de datos (base de datos, API, memoria)
- Aplica reglas de negocio y validaciones
- Retorna los datos procesados al Controlador

Paso 5 - Vista (Renderizado):

El Controlador selecciona la Vista adecuada y le pasa los datos del Modelo. La Vista:

- Toma los datos recibidos
- Genera el HTML dinámico (usando Razor u otro motor de plantillas)
- No contiene lógica de negocio, solo lógica de presentación
- El HTML renderizado se envía de vuelta al navegador como respuesta HTTP